

# AI Agents — Taxonomy and Ontologies

---

## AI Agents

---

### What Is an AI Agent?

An AI agent is a program that can think, decide, and act on its own to complete a goal — without you spelling out every step. You give it a task, and it figures out how to get there. That might mean searching the web, writing and running code, calling an API, or handing work off to another agent. It keeps going, adjusting as it goes, until the job is done. What makes it an "agent" rather than just a script is that it reasons about what to do next rather than following a fixed sequence.

### The Main Frameworks

There are four frameworks that most people in this space are working with, and each one takes a different approach to the same problem. LangChain was the first popular one — it lets you connect a language model to tools like search or databases through a chain of steps. It works well for simple, linear pipelines but quickly falls apart when you need anything that loops back on itself or branches depending on conditions. The abstraction layers also make it frustratingly hard to debug when things go wrong.

LangGraph came out of the LangChain team to fix exactly that. Instead of a straight line, it models workflows as a graph — so an agent can loop, retry a step, or take completely different paths depending on what happens. It is much more powerful, but that power comes with real complexity. Simple tasks become verbose to define, and you have to think carefully about how state is managed or things break in subtle ways.

CrewAI takes a different angle entirely — it is built around the idea of teams. You define a crew of specialized agents, give each one a role like researcher or writer or reviewer, and they work together like a small team would. It reads very naturally and is easy to set up for the right kind of task. The problem is that it handles failure poorly — if one agent in the crew produces bad output, the rest of the workflow often quietly degrades rather than stopping and raising an error.

AutoGen, built by Microsoft, has agents communicate with each other through conversation — actual back-and-forth messages. This makes it great for tasks where you want agents to debate an idea, critique each other's work, and refine it together. But for structured, repeatable workflows it is slow and unpredictable, because the "interface" between agents is natural language rather than a typed contract.

### Where They All Fall Short

What none of these frameworks were designed for is the harder stuff — security, physical deployment, and what happens when something goes wrong in a real production environment. They assume agents are trusted, environments are digital, and failures can always be recovered. That is fine for prototypes and demos, but it creates real gaps the moment you deploy agents on anything that actually matters.

## My Reasoning

What struck me most when looking at these frameworks is that they were all built to solve the capability problem — how do you get agents to do more, coordinate better, reason further? But almost nobody was asking the reliability and safety question at the same time. LangChain made agents accessible. LangGraph made them more flexible. CrewAI made multi-agent collaboration readable. AutoGen made collaborative reasoning possible. Each one moved the capability needle forward. But the industry is still catching up on what it means to run these things safely, predictably, and accountably at scale. The frameworks are impressive. The safety infrastructure around them is still mostly missing.

---

---

# Taxonomy of AI Agents

---

## Why Classification Matters

Saying something is "an AI agent" tells you almost nothing useful when you are trying to evaluate it, deploy it, or decide how much to trust it. You need to know how autonomous it is, whether it works alone or with others, and where it is actually running. Without a proper classification system, you cannot compare agents meaningfully, set the right safety requirements, or even have a clear conversation about what kind of system you are building. Taxonomy gives you that shared language.

## Three Dimensions That Actually Matter

The first dimension is coordination style — how the agent relates to other agents around it. At the simplest end, a solo agent works entirely alone. Sequential agents hand work to each other in order, like a relay. Parallel agents tackle different parts of a task at the same time. Hierarchical systems have one orchestrator agent delegating to specialized sub-agents. Peer-to-peer agents negotiate and coordinate as equals with no central controller. And swarm systems have many agents with no central coordination at all — order emerges from the interactions themselves.

The second dimension is autonomy level, which describes how much the agent does on its own versus how much a human stays in the loop. At L1 the human does everything and the AI just suggests. At L2 the AI handles small sub-tasks while the human stays in control. L3 is where the agent acts autonomously within defined limits and escalates only when something is outside its

scope. L4 agents handle almost everything on their own, with humans stepping in only for exceptions. L5 is full autonomy — the agent runs completely independently. Most real production systems today sit at L2 or L3. True L5 agents exist only in very narrow, tightly controlled domains.

The third dimension is deployment context — where the agent actually lives and acts. Digital agents operate entirely online, where actions are reversible and errors are generally fixable. Cyber-physical agents interact with physical systems but still have human override available. Embodied or fully physical agents — think robots or autonomous vehicles — act directly in the real world where mistakes cannot always be undone.

## **The Dimension Nobody Is Talking About**

All three of those dimensions matter, but there is a fourth one that the industry is largely ignoring: consequence severity. Two agents can look identical across coordination style, autonomy level, and deployment context — but one manages a music playlist and the other manages a hospital network. The risk profile is completely different. What needs to be part of any serious taxonomy is how bad the worst-case failure looks, how fast harm spreads, and whether it can be reversed. Without that, we are classifying agents without actually understanding their danger level.

## **My Reasoning**

Taxonomy feels like an academic exercise until you try to make a real decision about deploying an agent in a sensitive environment. Then you realize there is no common framework for comparing what you are building against what safety standards should apply. The three dimensions I covered are the starting point, but consequence severity is the one that would actually change how engineers design agents if it became a standard part of how we classify them. Right now, a startup building a low-stakes recommendation agent and a team building autonomous control for industrial infrastructure are both just building "agents." That gap in language is a gap in accountability.

---

# **Ontology of AI Agents**

---

## **What Ontology Means and Why It Matters**

Taxonomy tells you what type of agent something is. Ontology tells you what it is connected to — all the relationships between an agent and everything around it. If taxonomy is about labeling a node in a diagram, ontology is about drawing all the lines between nodes. This matters because most of the real problems in multi-agent systems — security failures, coordination breakdowns, bad decisions — do not happen inside a single agent. They happen at the connections. Understanding those relationships is how you build systems that can be reasoned about, audited, and held accountable.

# The Relationships That Define an Agent

An agent's relationship with memory is more layered than it first appears. There is what is in its current context window, which is short-term and disappears when the session ends. There is long-term memory stored in external databases and retrieved when relevant. There are episodic logs of what the agent has done before, and there is the general knowledge baked into the model itself. Most frameworks treat memory as a simple lookup, but what is missing is clear policy around what the agent can read versus write, and what gets kept versus discarded over time.

The agent's relationship with tools is where its actions become real. Tools are how it reaches into the outside world — search engines, code runners, APIs, file systems, email. What the current frameworks almost universally fail to model is the difference between a read-only action and one with side effects. Calling a search API and sending an email look the same to most frameworks, but one is harmless and one is not. Without modeling consequence at the tool level, agents can cause real-world harm through perfectly logical decision-making.

The environment relationship captures everything the agent perceives and affects. In digital systems this is relatively abstract. In physical systems the environment is constantly changing and reacting — often faster than the agent can reason. Most ontologies treat the environment as static between agent steps, which is acceptable for software tasks but completely wrong for any agent operating in the physical world.

Agent-to-agent relationships are where coordination happens, but also where trust becomes a serious question. Agents share memory, pass messages, delegate tasks, and consume each other's outputs. Current frameworks almost universally assume that all agents within a system trust each other by default. That assumption is a security vulnerability and a reliability problem — if one agent is compromised or simply produces bad output, a downstream agent that blindly trusts it will amplify the mistake.

The audit layer relationship is about what gets recorded. Every action an agent takes should leave a trace — what it did, why it decided to do it, and what happened as a result. In most systems this is bolted on as an afterthought rather than designed in. A proper audit relationship means logs are tamper-proof, detailed enough to reconstruct a decision after the fact, and stored somewhere that cannot be quietly erased.

Governance is the relationship between the agent and the rules it operates within — what it is allowed to do, what it must escalate, what data it can access. Today these policies are almost always set once at deployment and never change. What is missing is runtime governance — the ability to dynamically tighten or relax an agent's permissions based on context, risk level, or what it has been doing. An agent operating in a low-stakes situation and the same agent suddenly handling sensitive data should not have identical permissions.

## My Reasoning

What the ontology exercise made clear to me is that we talk about agents as if they are independent units, but they are really defined by their relationships. The agent itself is almost the

least interesting part — what matters is what it can remember, what it can do, who it trusts, and whether anyone is watching. The gaps I kept finding across every relationship — no trust modeling between agents, no consequence classification for tools, no runtime governance, no first-class audit design — are not small oversights. They are the exact gaps that get exploited when these systems are used in high-stakes environments. Getting the ontology right is not just an intellectual exercise. It is the prerequisite for building multi-agent systems that are actually safe to deploy.